

基于 FFT 的泊松方程快速求解器的硬件实现

李国燕¹, 顾军华², 宋庆增³, 陆益财², 周博君²

(1. 河北工业大学 电气工程学院, 天津 300130; 2. 河北工业大学 计算机科学与软件学院, 天津 300130;

3. 天津工业大学 计算机科学与软件学院, 天津 300387)

摘要: 针对传统的泊松方程求解算法执行效率低、功耗大, 很难满足实际需要的缺点, 设计了一种 FPGA 硬件平台的泊松方程快速求解器。设计采用软件与硬件结合的方式, 由软件执行控制复杂、计算量较小的任务, 而由硬件完成控制简单、计算量大的任务, 从而达到硬件加速的目的。在 FPGA 平台上, 独立设计的 FFT 协处理器可以流水和高度并行化的处理数据, 提高了求解器的性能。实验结果表明, 硬件实现的基于 FFT 的泊松方程快速求解器具有较高的计算性能和良好的可扩展性。

关键词: 泊松方程; FFT; FPGA; 快速求解器

Hardware Implementation of Fast Poisson Solver based on FFT

Li Guoyan¹, Gu Junhua², Song Qingzeng³, Lu Yicai², Zhou Bojun²

(1. School of Electrical Engineering and Automation, Hebei University of Technology, Tianjin 300130, China;

2. School of Computer Science and Software, Hebei University of Technology, Tianjin 300130, China;

3. School of Computer Science and Software Engineering, Tianjin Polytechnic University, Tianjin 300387, China)

Abstract: The traditional poisson solver is low efficiency and high power dissipation, it is difficult to meet the actual needs, this paper proposes a hardware design of poisson solver based on FPGA platform. In order to achieve the purpose of hardware acceleration, the design is based on hardware/software co-design. The code which including complexity control and less calculation is performed by the software part and the hardware part implements the code of enormous computation and simple control. In this paper, we design and implement a FFT co-processor on FPGA, it can pipeline and highly parallel process data. Final results illustrate that the hardware implementation of fast poisson solver based on FFT has high computing performance and good scalability.

Key words: poisson equation; FFT; FPGA; fast solve

0 引言

许多物理模型中最核心的就是泊松方程的求解, 泊松方程是数学中一个常见于静电学、机械工程和理论物理的椭圆型偏微分方程而一些其它类型方程(如弹性力学方程)的迭代解法往往也归结为对泊松方程的求解^[1]。在二维空间中, 泊松方程的表达式为:

$$\nabla^2 v(x, y) = g(x, y) \quad (1)$$

由于二维泊松方程是一个非齐次的二阶偏微分方程, 求解过程是复杂的, 甚至有时无法求解。理论上可以用数学物理方法中的格林公式求解, 但是该方法在实际应用时存在很大的局限性。在现代数值方法中, 有限差分法是最早也是比较适合的求解方法。有限差分法对于椭圆问题的逼近往往需要求解大规模的稀疏矩阵问题^[2]。实际应用中, 求解大型方程组最为常用的方法是逐步超松弛迭代法(SOR)求近似解^[3]。随着现代计算机的发展, 由许多偏微分方程的标准离散化得到的用于求解泊松方程的若干快速计算方法, 受到了人们的密切关注。Ogura 提出的利用 FFT 直接求解二维泊松方程的方法, 因其

效率极高而得到了广泛的应用。虽然, 基于 FFT 的偏微分方程求解方法精度高且速度较快, 但对于大数据量的处理, 利用软件实现仍然是困难的, 难以满足实时处理的要求。

基于 FFT 的泊松方程求解方法, 其理论基础是离散傅里叶变换, 具有天然的并行结构, 便于硬件电路实现。FPGA 作为一种广泛使用的可编程逻辑器件, 具有动态配置的灵活性、能实现流水并行设计的特点^[4]。因此, 本文基于 FPGA 的可重构计算平台设计和实现了一种泊松方程的 FFT 求解算法的硬件快速求解器, 重点设计了 FFT 模块, 并采用软硬件结合的方式, 将 Nios-II 软核处理器与 FFT 求解部分耦合在一起。实验结果表明, 设计的求解器具有较高的计算速度, 适合应用于大规模问题的求解。

1 算法分析

1.1 离散快速傅里叶变换与傅里叶矩阵

数字系统只能处理有限长的信号, 为此计算机上使用的是离散傅立叶变换(DFT)^[5-8]。对于一个有 N 个元素的有限长序列, DFT 定义为:

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{nk} \quad (2)$$

式中: $k = 0, 1, \dots, N-1$, W_N^k 是 DFT 的旋转因子, 满足

$$W_N = e^{-2\pi i/N} = \cos(\frac{2\pi}{N}) - i\sin(\frac{2\pi}{N}) \quad (3)$$

继而得到:

收稿日期:2012-08-15; 修回日期:2012-10-15。

基金项目:天津市应用基础及前沿技术研究计划项目(10JCYBJC00200)。

作者简介:李国燕(1984-),女,石家庄人,博士研究生,主要从事高性能计算方向的研究。

$$\begin{aligned} W_{2m+2}^{jk} &= e^{-2jk\pi i/(2m+2)} = e^{-jk\pi i} \\ W_{2m+2}^{jk} &= \cos(jk\pi h) - i\sin(jk\pi h) \end{aligned} \quad (4)$$

如果存在 $h_j = \sum_{k=1}^N W_N^{(j-1)(k-1)} g_k, j=1,2,\dots,N$, 并且 $h = [h_1, \dots, h_N]^T$ 是 $g = [g_1, \dots, g_N]^T$ 的 DFT, 则有 $h = D_N g$, 这里的 $D_N = (W_N^{(j-1)(k-1)})_{j,k=1}^N$, 那么 D_N 就叫做傅里叶矩阵, 例如 $W_4 = e^{-2\pi i/4} = \cos(\frac{\pi}{2}) - i\sin(\frac{\pi}{2}) = -i$

$$D_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}$$

同时, 分析可知 DFT 的旋转因子存在对称性和周期性, 如公式 (5) 和 (6) 所示:

$$(W_N^{nk})^* = W_N^{-nk} \quad (5)$$

$$W_N^{n(N+k)} = W_N^{kn} \quad (6)$$

$(W_N^{nk})^*$ 和 W_N^{kn} 是共轭关系。当 N 较大时, DFT 的计算量太大, 很难实时地处理问题, 因此引出了 FFT。FFT 是 DFT 的快速算法, 通过把多点的运算转换为小点的 DFT 计算, 可以将运算量减少到 $O(n \log n)$ 。

1.2 基于 FFT 的泊松方程求解

二维空间里, 一个域 $\Omega = \{(x, y) \mid r_1 \leq x \leq r_2, l_1 \leq y \leq l_2\}$ 中的泊松方程, 可以表示为

$$\frac{v_{j-1,k} - 2v_{j,k} + v_{j+1,k}}{\Delta x^2} + \frac{v_{j,k-1} - 2v_{j,k} + v_{j,k+1}}{\Delta y^2} = g(x_j, y_k) \quad (7)$$

式中: j 和 k 的取值范围为: $1 \leq j \leq m, 1 \leq k \leq n, m+1, n+1$ 分别是 x 和 y 方向上的总元素个数, Δx 和 Δy 是 x 和 y 的变化宽度, 且满足 $g(x_j, y_k) = g(r_1 + j\Delta x, l_1 + k\Delta y)$, $v_{j,k} \approx v(r_1 + j\Delta x, l_1 + k\Delta y)$ 当 $j=0, k=0, j=m+1, k=n+1$ 时, 取其边界值。

如果域 Ω 是一个方阵且有 $\Delta x = \Delta y = c$, 那么 x 方向和 y 方向有相同的元素, 即 $m = n$ 。若满足齐次狄利克雷边界条件, 即在 Ω 的偏微分域 $\partial \Omega$ 中有 $v(x, y) = 0$, 可以得到:

$$VT + TV = c^2 H \quad (8)$$

$$T = \begin{bmatrix} -2 & 1 & & \\ 1 & -2 & 1 & \\ & \dots & \dots & \dots \\ & & 1 & -2 & 1 \\ & & & 1 & -2 \end{bmatrix}, V = \begin{bmatrix} v_{1,1} & \dots & v_{1,n} \\ \dots & \dots & \dots \\ v_{n,1} & \dots & v_{n,n} \end{bmatrix},$$

$$H = \begin{bmatrix} g(x_1, y_1) & g(x_2, y_1) & \dots & g(x_n, y_1) \\ g(x_1, y_2) & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ g(x_1, y_n) & \dots & \dots & g(x_n, y_n) \end{bmatrix}$$

取 $c=1$, 则有 $VT + TV = H$ 。这时方程的求解步骤如下:

- (1) 先对 H 的每行进行离散正弦变换, 得 H_1 ;
- (2) 对 H_1 的每列进行离散正弦变换, 得到 H_2 ;
- (3) 对 H_2 做相应的线性变换, 得到 V_1 ;
- (4) 对 V_1 的每行进行离散正弦变换, 得到 V_2 ;
- (5) 再对 V_2 的每列进行离散正弦变换, 得到 V ;

分析可知, 对 H 进行 $4n$ 次离散正弦变换和一次线性变换可得结果。因此, 泊松方程可以通过计算定序序列的离散的正

弦变换和简单的线性变换求解, 而用 FFT 实现离散正弦变换^[9-10]。

2 系统设计与实现

本文采用软硬件结合的方式, 基于 FPGA 实现的系统主要包括 Nios-II 软核、FFT 协处理器和外部存储这 3 个模块, 并通过高速总线连接在一起。Nios-II 作为一个通用的可重构系统的微处理器, 控制主要的计算模块, 执行不适合硬件执行的代码; FFT 模块是系统最重要的模块, 完成泊松方程求解的主要计算, 对 FFT 模块的调用是关键; 外部存储器存储计算需要的数据。系统的整体架构如图 1 所示。

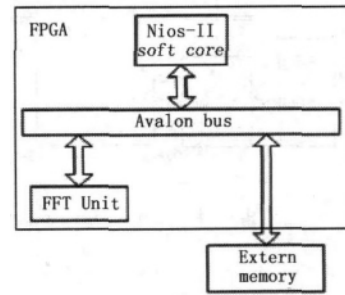


图 1 系统的体系结构

设计中确定 FFT 协处理器执行的代码是关键的一步, 划分不合理就会导致非常低的计算性能, 不能达到加速目的。针对大规模的求解系统, 泊松方程的求解过程需要进行 $4n$ 次 FFT 运算, 2 次线性变换, 和一些简单的控制运算。其中 90% 的计算时间都集中在 FFT 运算上, 而 FFT 算法在硬件上可以进行流水并行的方式进行, 非常适合硬件电路的方式执行。为了获得更高的加速比, 本文将 FFT 算法以硬件电路的方式执行, 其余控制部分则以软件形式在软核处理器上执行。

2.1 FFT 协处理器的设计与实现

本文设定 FFT 模块的点数为 64, 采用基 2 按频抽取算法的分解模式, 基 2 算法蝶形单元处理流程图如图 2 所示。对于 64 点 FFT 变换, 如果采用阵列处理结构会出现大规模的蝶形处理单元, 硬件资源占用量远高于并行处理结构和级联处理结构, 而并行处理结构的控制复杂高, 不利于硬件实现。因此, 本文采用级联处理结构。

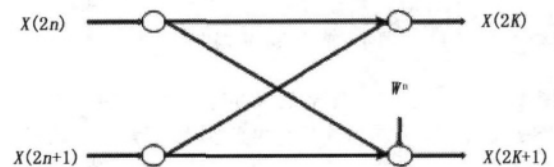


图 2 基 2 算法蝶形单元处理流程图

64 点级联 FFT 处理器共 6 级, 每级只有一个基 2 蝶形处理单元。采用 VHDL 对蝶形单元设计的各个管脚功能如下: Clk 是蝶形运算单元的时钟脉冲信号端; W_in 为旋转因子的实部; Wadd_in 与 Wsub_in 分别为旋转因子的实部与虚部的和与差。这 3 个管脚连接着存储旋转因子的 ROM, 与时钟使能信号同步传输数据。Areal_in、Aimag_in、Breal_in、Bimag_in 分别是节点数据的实部与虚部输入端口, 而 Dreal_in

out、Dimag_out、Ereal_out、Eimag_out 则分别为输出节点的实部与虚部输出端口。带有蝶形单元的每一级的结构如图 3 所示。

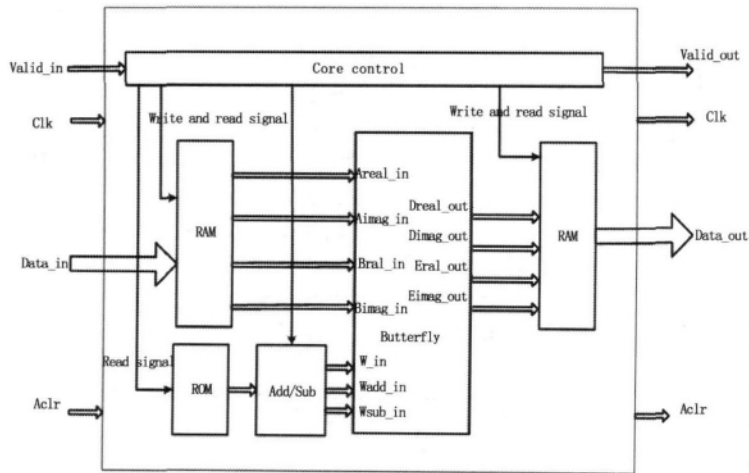


图 3 每一级电路结构示意图

由图 3 可以看出，每个单元从整体上只有 4 个输入与输出端口（最后一级除外），单元内部都有两个用于存储节点数据的 RAM 和一个存储旋转因子的 ROM，而这两个 RAM 都要分别和前一级和后一级共享。每级处理单元的蝶形单元用高效的复数乘法器进行运算节点与旋转因子的乘积，模块 Add/Sub 为蝶形单元实时产生相关的因子。这 5 个模块的时序控制由一个核心控制模块 Core control 完成。每级单元内的控制电路各有所能，完成不同时序控制，第一级核心控制模块电路的 VHDL 设计结构如图 4 所示。

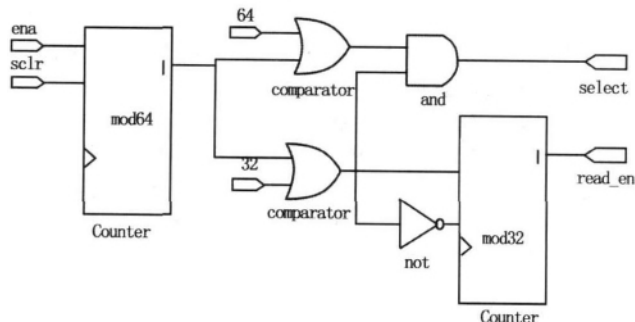


图 4 单元核心控制电路

图 4 中，ena 信号与外部的 valid_in 信号端口连接，sclr 信号与全局复位信号端口连接。电路实现过程中，计数器接收到使能信号并产生计数，计数加 1，产生的计数值传到两个比较器端口，在第一级处理单元中，计数数值分别与 32 和 64 相比。当计数数值小于 32 时 select 信号置为 '0'，否则为 '1'，它是处理后数据的控制信号。另外，在计数数值大于或等于 32 时，触发另一个计数器并产生计数，计数的数值给级内的 ROM，控制旋转因子地址的读取。

6 级的 FFT 处理器的结构图如图 5 所示，共包含 6 个蝶形单元与 7 个双口的 RAM，每个蝶形单元都与两个双口 RAM 相连，相邻的蝶形单元之间共享一个双口 RAM，并且每级都独立连接一个存储旋转因子的 ROM。经过 6 级的数据处理共

完成 64 点 FFT 变换运算。这样，可以通过多次调用该模块完成多次运算，提高运算性能。

2.2 Nios-II 软核控制结构的设计与分析

Nios-II 软核控制流程的设计如图 6 所示。其中，虚线框内的功能是在 Nios-II 软核上以软件形式完成的，而软核与 FFT 模块都由 FPGA 完成。

Nios-II 软核首先控制数据存储在 FPGA 外部的双口 RAM 中，并控制数据的第一轮读取。将数据传送到 FFT 模块中做第一轮 FFT 变换（一轮变换指对二维矩阵按行读取，并作 n 次 FFT 变换，将 n 次的值存入 RAM 中，然后再从 RAM 中读取数据并由转置函数单元作转置处理），转置后的数据再传送到 FFT 模块作第二轮 FFT 变换，之后的返回值存入 RAM 中。至此，完成数据的两轮正弦变换。Nios-II 软核第三轮从 RAM 中读取数据后，先由线性变换函数单元作相应的线性变换，再将其传送到 FFT 模块，返回值同样存入 RAM 中。最后一轮读取的数据，经过转置函数单元作转置处理后送入 FFT 模块。这样，求解器经过了四轮的数据处理后完成对方程的求解。

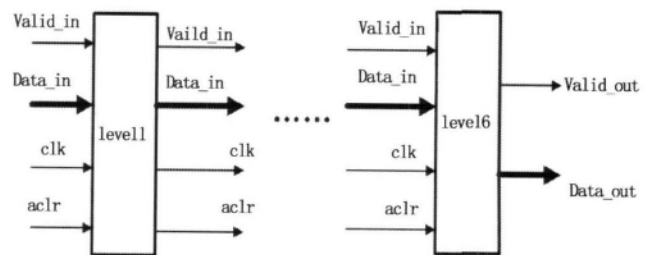


图 5 级联 FFT 处理器结构图

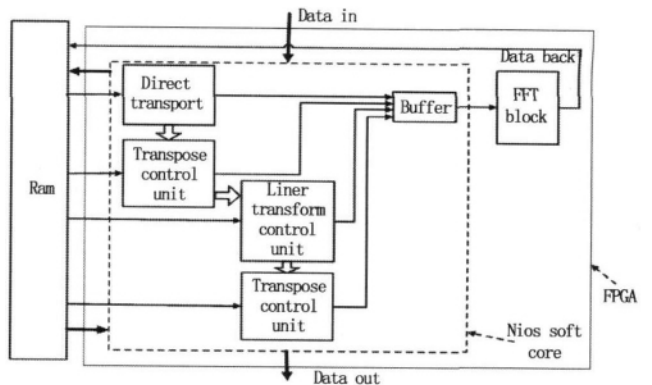


图 6 Nios-II 软核控制流程

3 系统仿真与实验结果分析

本文的硬件实验采用 Quartus-II 11.1 和 DSP-Builder 11.1 作为编程工具，并在 Altera 公司的 Cyclone-EP2C0F896C6 实现整个设计；软件实验在 Intel (R) Core (TM) i3 CPU M350 @ 2.27GHz 927MHZ, 1.92 GB 内存的 PC 机上用 MATLAB 作为编程工具。

本文设计的整个求解过程中系统需要 $4n$ 次调用 FFT 模块，而每次调用都会与外部存储器进行大量的数据交换，这就要求控制信号必须是准确的。在 Modelsim10.0c 环境下对 FFT 模块的设计进行仿真，仿真结果如图 7 所示。

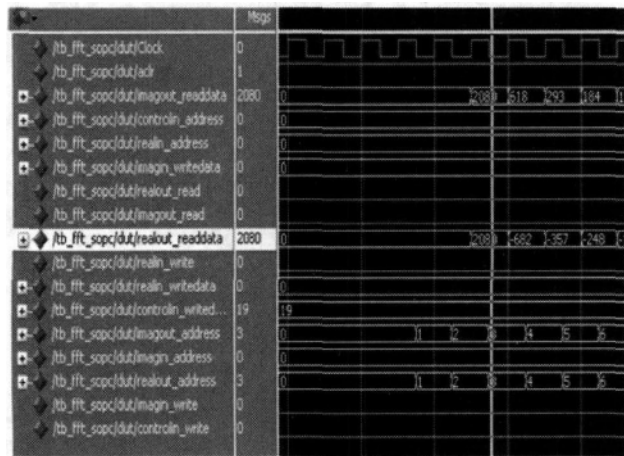


图 7 FFT 模块仿真效果

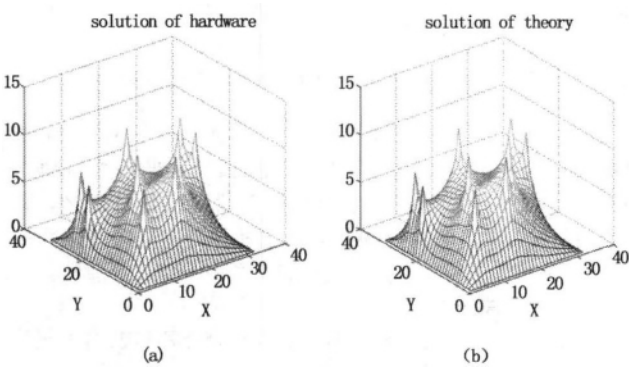
图 7 是对 FFT 模块做一维数据的测试，数据长度为 64，试验中输入数据的实部与虚部分别为 1 到 64 的等差数列。在一定时序延迟后，在图 7 中可观察到正确的数据输出。其仿真结果与理论值相符，说明设计是可靠的和稳定的。

FFT 模块的设计是整个求解器的核心部分，将设计的 FFT 模块耦合到整个求解器系统中，为了验证求解器的性能，实验算例分析如下：

在带电荷的静电场中， V 表示电压， ρ 表示电荷密度函数，由电荷在空间固有的重力势能而得到方程：

$$\nabla^2 V = -4\pi\rho \tag{9}$$

大量的电磁场问题需要求解方程 (9)。在满足狄利克雷边界条件的二维泊松方程中（定义在长度为 32 的平方区域），随机选取 ρ 中 10 个位置，(5, 14)、(25, 3)、(10, 7)、(16, 28)、(5, 5)、(19, 26)、(8, 17)、(20, 31)、(21, 2)、(23, 14)，并且设定这 10 个位置的值为 $(-4) * \pi$ ，而其他位置的值为 0。满足条件的 ρ 就是需要进行变换的矩阵 H ，利用本文设计的求解器的求解，可以得到解得电场分布如图 8 所示。



a. FPGA 硬件快速求解器结果分布图；b. 理论精确值分布图
图 8 泊松方程解分布图

直观上看，软硬件的计算效果相同。为了进一步验证基于 FPGA 的硬件系统的可靠性，可以采用最小均方误差 MSE 来衡量求解器的精度。实验求得其值为 0.0818，均方误差较小，由此可见本文设计的求解器具有很高的计算精度。

为了说明本设计具有较好的性能和较高的处理速度，表 1

和表 2 分别给出了设计中的硬件资源使用情况和软硬加速比。

表 1 硬件资源使用量

	使用的逻辑单元资源 (bit)	总的逻辑单元资源 (bit)	资源使用百分比
总的逻辑单元	8454	68416	12%
总的互连设备	7661	68416	11%
专用逻辑寄存器	6969	68416	10%
总的引脚	172	622	27%
总的内存	31060	1152000	2%
乘法器使用	150	300	50%
总的锁相环使用	1	4	25%

表 2 软件实现与硬件的加速比

Matlab 软件执行时间/s	基于 FPGA 设计执行时间/s	软硬件处理速度倍数比
9.1600e-004	6.2669e-005	14.6165

由表 1 可知，总的逻辑单元使用量仅为总器件的 12%。因此，设计使用的硬件资源较少，容易扩展，并且经过计算，求解器系统模型的运行频率能达到 65.3 MHz，性能较好。由表 2 可知，与软件求解相比，设计具有较高的处理速度，软硬加速比约为 14.6，较好的满足了对实时性和并行处理的要求。

4 总结

本文利用 FFT 的内在并行性，采用可重构的 FPGA 芯片实现了泊松方程的快速求解器。设计有效的应用了软硬件划分技术，分别以硬件和软件的方式执行，并独立针对性的设计了 FFT 模块，减少了硬件资源使用量，并提高了整个系统的计算性能。实验结果表明，在 FPGA 平台上，本文设计的基于 FFT 的泊松方程快速求解器具有较高的处理速度，能达到 14 倍以上的加速，可以应用于大规模问题的求解。

参考文献：

[1] Mary L. Boas, Mathematical Methods in the Physical Sciences [M], John Wiley & Sons, 3rd ed, 2006.

[2] 王忆锋, 唐利斌. 利用有限差分法和 MATLAB 矩阵运算直接求解二维泊松方程 [J]. 红外技术, 2010, 32 (4): 213-230.

[3] 孔凡轴. 用 FFT 法与 SOR 法求解二维泊松方程的比较 [J]. 气象科学, 1988, (1): 19-28.

[4] 石长振, 杨雪, 王贞松. 高性能并行 FFT 处理器的设计与实现 [J]. 计算机工程. 2012, 38 (2): 242-247.

[5] 华程. 基于 FPGA 的 FFT 算法研究与实现 [D]. 安徽: 安徽理工大学, 2011.

[6] Wang X D. A Time-Efficient FPGA-Based FFT Processor [J]. Lecture Notes in Electrical Engineering, Volume 140, 191-194, 2012.

[7] Loan C V. Computational Frameworks for the Fast Fourier Transform, SIAM, 1992.

[8] He S S, Torkelson M. A New Approach to Pipeline FFT Processor [C]. In Proceedings of IPPS '96, The 10th International, pages 766-770, Apr 1996.

[9] Hassan, Vytla F, Carletta L. Exploiting redundancy to solve the Poisson equation using local information Image Processing (ICIP) [C]. In Proceedings of the IEEE Conference on, International, pages 2689-2692, Nov. 2009.

[10] Ngho Jin Yee, Othman, M. Review on FPGA-Based Systolic Poisson Solver Computer Technology and Development [C]. IEEE Trans, on International Conference, pages 336-339, 2009.